Chapter 5

The Java Memory Model

This chapter summarizes the Java memory model and adapts it to the purposes of the PCPP course. The changes simply omit details on how the Java memory model must be enforced by implementations of the Java Virtual Machine. The definitions and reasoning described in this note are a consistent subset of the complete Java memory model[1] [9]. Therefore, the concepts and reasoning discussed here are applicable to analyze real Java software.

The Java memory model defines, given a concurrent Java program, the set of valid executions of the program. The Java Just-In-Time (JIT) compiler and Java Virtual Machine (JVM) guarantee that concurrent Java programs only produce executions allowed by the Java memory model. This guarantee is enforced in all hardware and operating systems supported by the Java runtime environment.

Precisely defining what are the possible executions of a program gives programmers the ability to predict the behavior of their programs. For instance, one of the main goals of the memory model is to determine, given an execution, what values are observed when performing a read on a variable.

Memory models in other programming languages We remark that a memory model is not a Java specific concept. Many popular programming languages have memory models, e.g., C/C++,[2] C#[3] or Go[4] just to mention a few. Generally, programming languages used for concurrent programming strive to precisely define a memory model, as it allows programmers to predict the behavior of their programs and ensure their correctness. For instance, the developers of the (rather new) programming language Rust are making efforts to define its memory model.[5] This is because, when a programming language with support for concurrency does not have a memory model, programmers cannot easily predict the behavior of their programs. The memory model for Java was the first to be completely formalized [9]. As a consequence, it is one of the most complete and mature memory models out there.

[1] Java Language Specification — Chapter 17. Threads and Locks (Java Memory Model).
[2] C++ memory model.
[3] C# memory model.
[4] Go memory model.
[5] Rust memory model (incomplete).

```
// Initial values
int x=0;y=0;
int a=0;b=0;

// Threads definition
Thread one = new Thread(() -> {
  a=1; // (1)
  x=b; // (2)
}).start();
Thread other = new Thread(() -> {
  b=1; // (3)
  y=a; // (4)
}).start();
```

Listing 5.1: Example of concurrent program with unexpected behavior.

5.1 Why do we need a memory model?

As mentioned above, a memory model allows programmers to predict the behavior of their concurrent programs. When no synchronization primitives are used in concurrent programs, they typically exhibit behavior that is considered paradoxical by programmers.

Example 1. Consider the program below—which we discussed in lecture 2.

Due to the lack of synchronization, the value of (`x`,`y`) after the execution of the program can be (1,1), (0,1), (1,0), but also (0,0). Most programmers would consider the first three cases as expected behaviors (due to the program interleavings). The result (1,1) can be generated by interleavings `(1),(3),(2),(4)` or `(1),(3),(4),(2)` or `(3),(1),(2),(4)` or `(3),(1),(4),(2)`. The result (0,1) and (1,0) by interleavings `(1),(2),(3),(4)` or `(3),(4),(1),(2)`, respectively. However, the result (0,0) would be unexpected for most programmers. The reason is that only interleavings `(2),(4),(1),(3)` or `(2),(4),(2),(4)` or `(4),(2),(1),(3)` or `(4),(2),(2),(4)` can produce that result. Note that these interleavings seem to contradict intra-thread order. That is, operation `(2)` occurs before `(1)` in thread `one`, and operation `(4)` occurs before operation `(3)`. This is considered unexpected or paradoxical by many programmers. □

The unexpected program executions illustrated in example 1 occur due to compiler and hardware optimizations built-in the JIT compiler, the JVM or the computer's CPU. Without these optimizations, running concurrent programs would not be as efficient as it is today.

5.2 Sequential consistency

What we have called "expected behavior" in the previous section is known as *sequential consistency*. The notion of sequential consistency was first introduced by Leslie Lamport in the late 70s [7]. In a nutshell, a concurrent execution is sequentially consistent iff i) it preserves program order (i.e., intra-thread consistency); and ii) operations appear to occur one at a time. For now, this intuition is all you need to know about sequential consistency. In the coming weeks, we will cover sequential consistency in detail. The mandatory readings for sequential consistency include Leslie Lamport's paper [7] and Section 3.3 in Herlihy.

Note that, if a runtime environment—such as the JVM—would enforce sequential consistency, then the interleavings producing (0,0) in example 1 would not be allowed. However, *ensuring sequential consistency for all concurrent programs would require disabling compilation and CPU optimizations that increase performance.*

As we will see below, the Java memory model allows programmers to use synchronization operations to control the behavior of their concurrent programs without the need to globally disable compilation or CPU optimizations.

5.3 Basic definitions

In this section, we provide the definitions of Java memory model that we will use in the course as well as examples on how to use them to reason about concurrent Java programs.

As mentioned earlier, the content in this section corresponds to a subset of the definitions in the complete Java memory model. This subset contains the required definitions to establish correctness of Java concurrent programs according to the Java memory model (cf. section 5.4), and omits details on how the Java memory model is enforced by the Java virtual machine.

5.3.1 Actions

Let $\mathcal{O}$ denote a set of *operations* and $\mathcal{T}$ a set of *threads*. Given a thread $t \in \mathcal{T}$ and an operation $o \in \mathcal{O}$, and *action*, denoted as $t(o)$, represents the execution of operation o by thread t. We use $\mathcal{A} \subseteq \mathcal{T} \times \mathcal{O}$ to denote the set of all actions.

We consider three types of actions:

Variable accesses These actions correspond to read and write accesses on program variables. For instance, `x=1` is a write access on variable `x` and, when evaluating the program term `x>0`, a read access on variable `x` is executed.

Synchronization actions These actions correspond to synchronization primitives, which includes:

- `lock()`/`unlock()` method calls on locks. Recall that other synchronization primitives such as monitors or semaphores internally use locks in their implementation.
- Write/read accesses on `volatile` variables.
- Operations that start a thread (such as `start()`) or detect that a thread has terminated (such as `join()`).

Other actions This category of actions corresponds to any actions not included in the other two categories. Actions in this category are often uninteresting for concurrency purposes.

Generally, an operation o refers to a program statement. However, actions executing an operation may have multiple categorizations. For example, in listing 5.1, the operation `(2)` corresponds to program statement `x=b`. When a thread executes this operation, the action classifies as a read access on variable `b` and a write access to variable `x`. For operations on method calls, sometimes we might abstract from the implementation of the method and simply assume whether it performs a read or write access on a variable. For instance, given an object `s` of type `Set`, the action modeling the execution of `s.add(x)` can be categorized as a write

access on `s` and a read access on `x`. If you use this type of abstractions in your reasoning, you must explicitly mention it.

5.3.2 Executions

An *execution* $e \in \mathcal{E}$ is a sequence of actions $e = a_1, a_2, \ldots$ where $a_i \in \mathcal{A}$. We use $a \in e$ to denote that action a occurs in execution e. The execution specifies the order in which actions are executed by the concurrent program. Recall that a single concurrent program may lead to multiple possible executions. Note also that this definition of execution is the same as the one for interleavings (introduced in lecture 1). In fact, the syntax for executions is similar to that of interleavings.

Example 2. Note that in the interleavings in example 1, we have omitted the thread identifier due to the lack of ambiguity. Consider the execution that results in `x==0` and `y==1` and let $t1, t2$ denote threads `one` and `other`, respectively. To precisely follow the syntax for executions we write $t1(1), t1(2), t2(3), t2(4)$ for said interleaving. □

5.3.3 Program order

Program order defines intra-thread execution order. That is, the execution order of operations within the thread. You may think of the program order as the execution order of the operations of the thread, when the thread is executed in isolation.

Definition 10. Given two actions a, b executed by the same thread t, we say a occurs before b according to program order iff a would be performed before b when t is executed sequentially and in isolation (i.e., without any other concurrent threads being part of the execution). □

The program order relation is a total order among the operations of each thread.

Example 3. Consider again the concurrent program in example 1. The program has two threads. We discuss the operations ordered by program order for each thread separately.

For thread `one`, denoted as $t1$, the program order relation includes the pair of actions $(t1(1), t1(2))$. Intuitively, this means that operation `(1)` must (appear) to occur before `(2)` within the thread. Note that, since these two operations are not data dependent, the result of executing `(1),(2)` or `(2),(1)` in a sequential execution, produces the same final value for `a` and `x`. Thus, the JIT compiler, JVM or CPU can reorder these operations. However, as we will see later (cf. section 5.3.6), at the Java memory model level only executions consistent with program order are valid. This is not a limitation, but a huge advantage. This means that, as programmers, we do not need to worry about compiler or CPU optimizations. We only need to consider valid executions at the memory model level, and the JVM implementation takes care of only allowing optimizations that are consistent with the set of valid executions for the program.

Analogously, for thread `other`, denoted as $t2$, program order includes the pair $(t2(3), t2(4))$. □

5.3.4 Happens-before order

The *happens-before order* establishes an order between actions of a concurrent program.

Definition 11. Given an execution $e \in \mathcal{E}$, and actions $a, b \in e$, we say that a *happens-before* b, denoted as $a \rightarrow b$, iff

- *Program order rule:* Actions a, b are executed by the same thread, and a appears before b in program order (cf. section 5.3.3).

- *Monitor rule:* Action a is an `unlock()` operation on a monitor,[6] and b is a subsequent `lock()` operation on the same monitor.

- *Volatile rule:* Action a is a write access on a `volatile` variable, and b is a subsequent read access on the same `volatile` variable.

- *Default init rule:* Action a writes the default value of a variable during object initialization and b is the first action of any other thread. The default value for `boolean` variables is false, for `int`/`float`/`double` is 0 and for non-primitive types (such as objects) is `null`.

- *Final/static field init rule:* Action a writes the initial value of a `final`/`static` variable during object initialization and b is the first action of any other thread.

- *Thread start rule:* Action a is a call to `start()` and b is the first action in the started thread.

- *Thread termination rule:* Action a is the last action executed by a thread and b is a call to `join()`, by any other thread, to wait for the thread executing a.

- *Transitive rule:* Given an action $c \in e$, if $a \to c$ and $c \to b$, then $a \to b$. □

The happens-before order is a partial order among the actions of an execution. Given a set of happens-before pairs that are related by transitivity, such as $\{a_1 \to a_2, a_2 \to a_3, a_3 \to a_4, \ldots\}$, we use the syntax $a_1 \to a_2 \to a_3 \to a_4 \to \ldots$ for brevity and convenience.

5.3.5 Synchronization order

Due to non-determinism in the possible executions of a program, synchronization actions may synchronize in different ways. The *synchronization order* relation establishes the legal ways in which synchronization actions may occur in an execution. Synchronization order is important as it characterizes the possible executions of a program (cf. section 5.3.8).

Definition 12. Given an execution $e \in \mathcal{E}$, a *synchronization order* is a total order between all synchronization actions in e such that:

- The synchronization order is consistent with mutual exclusion. This implies that `lock()` and `unlock()` operations are correctly nested.

- The synchronization order is consistent with happens-before order (cf. section 5.3.4). □

Each execution $e \in \mathcal{E}$ has a synchronization order associated to it.

[6] Here we use "monitor" as an abstract term, as the Java bytecode operations for synchronization use this term (cf. `https://docs.oracle.com/javase/specs/jvms/se17/html/jvms-6.html`). However, the rule applies to the corresponding operations in other synchronization objects such as locks, semaphores, barriers, etc.

5.3.6 Well-formed executions

Now we are ready to define what a *valid* execution is according to the Java memory model. To this end, we define *well-formed* executions.

Definition 13. An execution $e \in \mathcal{E}$ is *well-formed* iff it is consistent with program order (cf. section 5.3.3), happens-before order (cf. section 5.3.4) and synchronization order (cf. section 5.3.5). □

Most importantly, given a concurrent Java program, the JVM only produces well-formed executions [9]. Thus, we can use the Java memory model to understand precisely the behavior of a Java concurrent program.

5.3.7 Visibility

Establishing a happens-before order between read and write accesses ensures visibility. This is enforced by the JVM. Programmers do not need to worry about the how the hardware ensures data consistency in the different levels of the memory hierarchy. For programmers, it suffices to establish a happens-before order between variables of interest at the programming language level.

Generally, given an execution $e \in \mathcal{E}$ and actions $r_x, w_x \in e$ where r_x is read access on a variable `x` and w_x a write access on `x`, if $w_x \rightarrow r_x$ holds, then the effect of w_x is visible to r_x.

Note that showing $w_x \rightarrow r_x$ that does guarantee that the value written by w_x is the one that r_x reads. This is because the happens-before order can relate more than one write access to the read in the same execution. For instance, consider the writes $w'_x, w''_w \in e$, then we might have that $w_x \rightarrow r_x$, $w'_x \rightarrow r_x$ and $w''_x \rightarrow r_x$ all hold in execution e. In this situation, the order of actions in the execution determines the value read by r_x.

Given an execution $e \in \mathcal{E}$, if we can show that $w_x \rightarrow r_x$ holds *and* for any other write access $w'_x \in e$ it holds that $w'_x \rightarrow w_x$, then it is guaranteed that r_x reads the value written by w_x. This is because we have showed that there cannot be any write access w'_x executed in between w_x and r_x.

5.3.8 A complete example

Here we discuss a complete example to illustrate the use of the Java memory model to predict the value of program variables after the execution of the program. The example is purposely verbose. The goal of the example is for you to see how to apply the definitions above. In practice, reasoning may be more compact; (see, e.g., examples 6 and 7). However, remember that lack of precision can lead to missing details, which may lead to incorrect conclusions.

Example 4. Consider the concurrent program in listing 5.2, which is a modification of the program in example 1. Concretely, we have added a lock to ensure mutual exclusion on operations performed by each thread. In this example, we use the Java memory model to show that the value of (`x`, `y`) after the execution of the program can only be (0,1) or (1,0)—i.e., the values (1,1) and (0,0) are not possible.

To show that only the values (`x`, `y`) after the execution of the program can only be (0,1) or (1,0), our reasoning must encompass all well-formed executions (cf. section 5.3.6) of this program. In particular, we must derive the pairs in the happens-before orders of all well-formed executions of the program.

```
// Initial values
Lock l = new ReentrantLock();
int x=0;y=0;
int a=0;b=0;

// Threads definition
Thread one = new Thread(() -> {
  l.lock();   // (1)
  a=1;        // (2)
  x=b;        // (3)
  l.unlock(); // (4)
}).start();
Thread other = new Thread(() -> {
  l.lock();   // (5)
  b=1;        // (6)
  y=a;        // (7)
  l.unlock(); // (8)
}).start();
```

Listing 5.2: Program using synchronization to remove unexpected behavior.

First, let us identify the happens-before pairs originating from the program order rule. This pairs must hold for all possible executions independently of synchronization order. For thread `one` we have $t1(1) \rightarrow t1(2)$, $t1(2) \rightarrow t1(3)$ and $t1(3) \rightarrow t1(4)$. For thread `other` we have $t2(5) \rightarrow t2(6)$, $t2(6) \rightarrow t2(7)$ and $t2(7) \rightarrow t2(8)$. Thus, the happens-before order for each thread is defined as follows:

$$HB_{po}^{t1} = \{t1(1) \rightarrow t1(2), t1(2) \rightarrow t1(3), t1(3) \rightarrow t1(4), \ldots\},$$

$$HB_{po}^{t2} = \{t2(5) \rightarrow t2(6), t2(6) \rightarrow t2(7), t2(7) \rightarrow t2(8), \ldots\}.$$

We use ... to denote that anything that can be derived by applying the transitive rule. For instance, the set HB_{po}^{t1} also contains the pairs $t1(1) \rightarrow t1(3)$ or $t1(1) \rightarrow t1(4)$.

There exists also an implicit main thread, denoted as m, which performs the initialization of the variables(`x`, `y`, `a`, `b` and `l`) and starts the threads. Let us denote the initialization actions as $m(init(x))$, $m(init(y))$, $m(init(a))$, $m(init(b))$, and $m(init(l))$. Let us denote the start thread operations as $m(start(t1))$, $m(start(t2))$. Then, the happens-before order for the main thread is defined as:

$$\begin{aligned} HB_{po}^{m} = &\{m(init(l)) \rightarrow m(init(x)), m(init(x)) \rightarrow m(init(y)), m(init(y)) \rightarrow m(init(a)), \\ &m(init(a)) \rightarrow m(init(b)), m(init(b)) \rightarrow m(start(t1)), m(start(t1)) \rightarrow m(start(t2)), \ldots\}. \end{aligned}$$

In summary, the set of happens-before pairs from program order rules is defined as:

$$HB_{po} = HB_{po}^{m} \cup HB_{po}^{t1} \cup HB_{po}^{t2}.$$

The second step is to identify happens-before pairs involving initialization operations. Recall that, by the thread start rule, the action executing `start()` happens-before the first

action within the started thread (cf. section 5.3.4). Then, for all program executions, the happens-before order includes the following pairs:

$$HB_{init} = \{m(start(t1)) \rightarrow t1(1), m(start(t2)) \rightarrow t2(5)\}.$$

Third, we must define all possible synchronization orders. Synchronization orders characterize the possible executions of a concurrent program. To this end, let us identify synchronization actions. The program in listing 5.2 contains 6 synchronization actions: $t1(1)$, $t1(4)$, $t2(5)$, $t2(8)$, $m(start(t1))$ and $m(start(t2))$, i.e., actions related to `lock()`/`unlock()` on lock `l` and thread start actions. Let us use S to denote the set of synchronization orders for this program. There are 4 possible synchronization orders for this program:

$$\begin{aligned} S = \{ & \\ & (m(start(t1)), m(start(t2)), t1(1), t1(4), t2(5), t2(8)), \\ & (m(start(t1)), t1(1), m(start(t2)), t1(4), t2(5), t2(8)), \\ & (m(start(t1)), t1(1), t1(4), m(start(t2)), t2(5), t2(8)), \\ & (m(start(t1)), m(start(t2)), t2(5), t2(8), t1(1), t1(4)) \\ \}. & \end{aligned}$$

Now we must argue that there are no other ordering of synchronization actions that would be consistent with program order and mutual exclusion. There are only two nesting of synchronization operations consistent with mutual exclusion either $t1$ acquires the lock before $t2$ or vice-versa. The first 3 cases correspond to the situation when $t1$ acquires the lock before $t2$. In these cases, it must hold that $t1(4) \rightarrow t2(5)$, and they must be consistent with program order, i.e., $t1(1) \rightarrow t1(4)$, $t2(5) \rightarrow t2(8)$ and $m(start(t1)) \rightarrow m(start(t2))$. We have three cases for this situation depending on when $m(start(t2))$ occurs. Due to the thread start rule, this action must happen before $t2(5)$. Thus, there are three possibilities that are consistent with program order; either $m(start(t2))$ occurs: between $m(start(t1))$ and $t1(1)$, or between $t1(1)$ and $t1(4)$, or between $t1(4)$ and $t2(5)$ The last synchronization order in S corresponds to the case when $t2$ acquires the lock before $t1$. In this case, it must hold that $t2(8) \rightarrow t1(1)$. Since, due to program order, we must have that $m(start(t1)) \rightarrow m(start(t2))$ and $m(start(t2)) \rightarrow t2(5)$, then there is only one possible order consistent with program order.

The next step is to derive the pairs of actions ordered by happens-before in the possible executions of this program. Note that each synchronization order gives rise to a possible execution. However, recall that HB_{po} and HB_{init} hold for all executions. This means that we only need to add the pairs of actions in the synchronization orders that are not present in HB_{po}or HB_{init}. This only gives two cases: when $t1$ acquires the lock first or when $t2$ acquires the lock first.[7]

Let us first consider the executions where $t1$ acquires the lock first. The only missing happens-before pair—in the 3 synchronization orders for this case—that is not present in HB_{po} and HB_{init} is $t1(4) \rightarrow t2(5)$. This pair is derived by applying the monitor rule (cf. section 5.3.4). Therefore, the complete set of pairs ordered by happens-before in this execution,

[7]This is a very common situation in programs with locks. When working on the exercises, you may omit listing all synchronization orders as long as you can determine the distinct happens-before orders that capture all possible executions.

denoted by HB^1 is

$$HB^1 = HB_{init} \cup HB_{po} \cup \{t1(4) \rightarrow t2(5)\} \cup \{\ldots\}.$$

As before, the last $\{\ldots\}$ refers to the pairs obtained by transitivity from the complete set $HB_{init} \cup HB_{po} \cup \{t1(4) \rightarrow t2(5)\}$.

For the case where $t2$ acquires the lock first, the reasoning is analogous. Thus, the only missing happens-before pair is $t2(8) \rightarrow t1(1)$. Therefore,

$$HB^1 = HB_{init} \cup HB_{po} \cup \{t1(8) \rightarrow t2(1)\} \cup \{\ldots\}.$$

In summary, HB^1 and HB^2 capture the only two possible sets of happens-before orders for the executions of this program. We will use these happens-before orders to show that the only possible final values for `x` and `y` are (0,1) and (1,0).

We show that in HB^1 the final value of (`x`, `y`) must be (0,1).

First we show that the value of `y` must be 1 after execution. To this end, we must show that the following holds

$$m(init(y)) \rightarrow m(init(a)) \rightarrow t1(2) \rightarrow t2(7).$$

This ensures that the last value written in `y` is 1 (cf. section 5.3.7). The pair $m(init(y)) \rightarrow m(init(a))$ follows trivially as it is in HB_{po}. Then, we must show $t1(2) \rightarrow t2(7)$ and $m(init(a)) \rightarrow t1(2)$ are in HB^1. The pair $t1(2) \rightarrow t2(7)$ follows by transitivity from the following pairs:

$t1(2) \rightarrow t1(3)$	in HB_{po}
$t1(3) \rightarrow t1(4)$	in HB_{po}
$t1(4) \rightarrow t2(5)$	from synchronization order in HB^1
$t2(5) \rightarrow t2(6)$	in HB_{po}
$t2(6) \rightarrow t2(7)$	in HB_{po}.

The pair $m(init(a)) \rightarrow t1(2)$ follows by transitivity from the following pairs:

$m(init(y)) \rightarrow m(start(t1))$	in HB_{po}
$m(start(t1)) \rightarrow t1(1)$	in HB_{init}
$t1(1) \rightarrow t1(2)$	in HB_{po}.

Thus, the value of `y` after this execution is guaranteed to be 1.

Second we show that the value of `x` must be 0 after execution. To this end, we must show that the following holds

$$m(init(x)) \rightarrow m(init(b)) \rightarrow t1(3) \rightarrow t2(6).$$

As before, this ensures that the last value written in `x` is 0 (cf. section 5.3.7). The pair $m(init(x)) \rightarrow m(init(b))$ follows trivially as it is in HB_{po}. Then, we must show that $t1(3) \rightarrow t2(6)$ and $m(init(b) \rightarrow t1(3))$ are in HB^1. The pair $t1(3) \rightarrow t2(6)$ follows by transitivity,

$t1(3) \rightarrow t1(4)$	in HB_{po}
$t1(4) \rightarrow t2(5)$	from synchronization order in HB^1
$t2(5) \rightarrow t2(6)$	in HB_{po}.

The pair $m(init(b)) \rightarrow t1(3)$ follows by transitivity,

$$\begin{aligned} &m(init(b)) \rightarrow m(start(t1)) && \text{in } HB_{po} \\ &m(start(t1)) \rightarrow t1(1) && \text{in } HB_{init} \\ &t1(1) \rightarrow t1(2) && \text{in } HB_{po} \\ &t2(2) \rightarrow t2(3) && \text{in } HB_{po}. \end{aligned}$$

Thus, the value of `x` after this execution is guaranteed to be 0.

The same reasoning can be used to show that for HB^2 the final value of (`x`, `y`) must be (1,0). □

5.4 Correct concurrent Java programs

The Java memory model uses data race freedom as the notion of correctness for a concurrent Java program. A concurrent program whose possible executions are data race free is known as correctly synchronized. In what follows we precisely define this notion.

5.4.1 Conflicting actions

We start by defining the actions in an execution that can be potentially leading to a data race. These actions are known as *conflicting*.

Definition 14. Given an execution $e \in \mathcal{E}$, we say that actions $a, b \in e$ are *conflicting* iff they are accesses on the same (non-`volatile`) variable and at least one of them is a write access. □

Example 5. Consider the program listing 5.2 from example 4. The actions $t1(2)$ and $t2(7)$—present in all the executions of the program—are an example of conflicting actions. This is because $t1(1)$ performs a write access on variable `a` and $t2(7)$ performs a read access on the same variable. □

We remark that variable accesses on `volatile` variables are never conflicting.

5.4.2 Data races

Now we are ready to introduce the definition of data race according to the Java memory model.

Definition 15. Given an execution $e \in \mathcal{E}$, there exist a data race between actions $a, b \in e$ iff a, b are *conflicting* and they are not ordered by happens-before. □

Example 6. Consider again the program listing 5.2 from example 4. As mentioned in example 5, actions $t1(2)$ and $t2(7)$ are conflicting. However, it holds that $t1(2) \rightarrow t2(7) \in HB^1$ and $t2(7) \rightarrow t1(2) \in HB^2$ (cf. example 4). Therefore, this actions do not lead to a data race in any of the executions of the program. □

Example 7. Consider now the program listing 5.1 from example 1. The happens-before order for all executions of the program only contains pairs of actions based on the program order

rule, and the synchronization actions from the thread start rule. That is,

$$
\begin{aligned}
&HB^{m}_{po} = m(init(x)) \rightarrow m(init(y)) \rightarrow m(init(a)) \rightarrow m(init(b)),\\
&HB^{t1}_{po} = \{t1(1) \rightarrow t1(2)\},\\
&HB^{t2}_{po} = \{t2(3) \rightarrow t2(4)\}, \text{and}\\
&HB_{init} = \{m(start(t1)) \rightarrow t1(1), m(start(t2)) \rightarrow t2(5)\}
\end{aligned}
$$

such that HB for all executions of this program is the transitive closure of $HB^{m}_{po} \cup HB^{t1}_{po} \cup HB^{t2}_{po} \cup HB_{init}$. However, due to the lack of synchronization actions between threads $t1$ and $t2$, it trivially follows that neither $t1(2) \rightarrow t2(7)$ nor $t2(7) \rightarrow t1(2)$ are in HB. Consequently, this program contains (at least) a data race. In other words, the program is not data race free. □

5.4.3 Correctly synchronized programs

Finally, we are ready to precisely define correctness for Java concurrent programs as stated in the Java memory model.

Definition 16. A program is *correctly synchronized* iff none of its executions contains data races. □

Example 8. Given the reasoning in example 7, we can conclude that the program in listing 5.1 is not correctly synchronized. □

Example 9. Consider now the program listing 5.2 from example 4. We show here that it is correctly synchronized, i.e., no execution contains data races.

First, we identify all conflicting actions in the executions of the program. We group them by variable:

- For variable `a` we have:
  - $(m(init(a)), t1(2))$
  - $(m(init(a)), t2(7))$
  - $(t1(2), t2(7))$
- For variable `b` we have:
  - $(m(init(b)), t1(3))$
  - $(m(init(b)), t2(6))$
  - $(t1(3), t2(6))$
- For variable `x` we have only $(m(init(x)), t1(3))$
- For variable `y` we have only $(m(init(y)), t2(7))$

Recall that this program has two possible sets of happens-before orders for all actions HB^1, HB^2, which correspond to whether the lock is first acquired by $t1$ or $t2$, respectively. Note that in both happens-before orders it holds that initialization operations happen-before

the operations within each thread. That is, by the program order and the thread start rules, HB^1 and HB^2 for any action a within $t1$ and $t2$ we have

$$m(init(x)) \rightarrow m(init(y)) \rightarrow m(init(a)) \rightarrow m(init(b)) \rightarrow a$$

This proves that there cannot be data races in conflicting pairs of actions involving an initialization operation. Thus, we are left to show that either $t1(2) \rightarrow t2(7)$ or $t2(7) \rightarrow t1(2)$ is in HB^1, HB^2 and similarly for $(t1(3), t2(6))$. The reason why we must consider both directions is that the actions must be ordered by happens-before in all possible executions. Since depending on what thread acquires the lock first, we might have a different order, it is necessary to consider both directions. In example 4 we have already shown that $t1(2) \rightarrow t2(7)$ is in HB^1. Using the pairs of actions from program order and using the happens-before pair $t2(8) \rightarrow t1(1)$ from the monitor rule and the synchronization order for HB^2, we have $t2(7) \rightarrow t2(8) \rightarrow t1(1) \rightarrow t1(2)$ is in HB^2. Analogous reasoning can be used to show that $t1(3) \rightarrow t2(6) \in HB^1$ and $t2(6) \rightarrow t1(3) \in HB^2$. Thus, we have shown that all conflicting actions are ordered by happens-before in all program executions. Consequently, we can conclude no execution leads to data races, which implies that the program is correctly synchronized. □

A remark on sequential consistency

The Java memory model ensures that all executions of correctly synchronized programs are sequentially consistent (cf. section 5.2). This is very useful for programmers, as it ensures that runtime, compiler or CPU optimizations cannot produce unexpected executions. In other words, programmers can ignore these optimizations when writing concurrent programs, and simply focus on the rules of the Java memory model.

5.5 A few more examples

In this section, we show how to use happens-before order rules and reasoning in situations distinct from the ones in previous examples. Nevertheless, we remark that the complete example in section 5.3.8 illustrates how to use all the core elements of the Java memory model.

5.5.1 Example with volatile variables

Example 10. Consider a modification of the program in listing 5.1 from example 1 where we declare variables `a`, `b` as `volatile`. In lecture 2, we discussed that this modification does not allow executions where the final value of (`x`, `y`) is (0,0). We can use the Java memory model to argue why this is the case. In particular, we show that the modified program is correctly synchronized. As a consequence of this, only sequentially consistent executions are valid, which—as mentioned in section 5.2—does not allow executions producing (0,0).

To show that the modified program is correctly synchronized, we must show that no execution contains data races (cf. section 5.4.3).

Before we argue about the happens-before order sets that capture all execution of this program, let us identify the conflicting actions in the program. Recall that, by definition, actions involving read/write accesses on `volatile` variables are not conflicting (cf. section 5.4.1). Thus, the only pairs of conflicting actions in the program are: $m(init(x), t1(2))$ and $m(init(y), t2(4))$.

The happens-before order sets containing actions pairs from the program order and thread start rule (HB_{po} and HB_{init}) are defined as follows:

$$HB^{m}_{po} = m(init(x)) \rightarrow m(init(y)) \rightarrow m(init(a)) \rightarrow m(init(b)) \rightarrow m(start(t1)) \rightarrow m(start(t2))$$
$$HB^{t1}_{po} = t1(1) \rightarrow t1(2)$$
$$HB^{t2}_{po} = t2(3) \rightarrow t2(4)$$
$$HB_{init} = \{m(start(t1)) \rightarrow t1(1), m(start(t2)) \rightarrow t2(3)\}$$

As before, we use HB to denote the transitive closure of $HB^{m}_{po} \cup HB^{t1}_{po} \cup HB^{t2}_{po} \cup HB_{init}$. Typically, at this point, we should identify all possible synchronization orders for this program. Given that actions involving read/write accesses on `volatile` variables are synchronization actions, we must consider all possible total orders of the actions $m(start(t1))$, $m(start(t2))$,$t1(1)$, $t1(2)$, $t2(3)$ and $t2(4)$. However, recall that all synchronization orders must be consistent with $HB_{po}$and HB_{init}. Therefore, for all executions, the action pairs in HB must hold.

We show that $m(init(x) \rightarrow t1(2))$ and $m(init(y) \rightarrow t2(4))$ hold in HB.

$$\begin{array}{ll} m(init(x)) \rightarrow m(start(t1)) & \text{in } HB_{po} \\ m(start(t1)) \rightarrow t1(1) & \text{in } HB_{init} \\ t1(1) \rightarrow t1(2) & \text{in } HB_{po} \end{array}$$

and

$$\begin{array}{ll} m(init(y)) \rightarrow m(start(t2)) & \text{in } HB_{po} \\ m(start(t2)) \rightarrow t2(3) & \text{in } HB_{init} \\ t2(3) \rightarrow t2(4) & \text{in } HB_{po}. \end{array}$$

Thus, we have shown that the conflicting actions in the program are ordered by happens-before for all executions. As a consequence, the program is data race free, which, in turn, implies that the program is correctly synchronized.

Since the program is correctly synchronized, all executions are sequentially consistent. Thus, no execution ends with the value (0,0) for variables (`x`,`y`); as producing these values requires violating program order. □

5.5.2 Example with conditional statements

The presence of conditional statements—such as `if` or `while`—impacts the set of action pairs ordered by happens-before. This impact is not mechanically derived from the core elements of the Java memory models discussed in section 5.3. When determining the set of executions in a program with conditional statements, we must analyze whether the predicates in the conditional statements evaluate to true or not. It is possible that there will be different sets of happens-before orders depending on the evaluation of the predicate. In what follows, we describe an example illustrating some of these subtleties.

Example 11. Consider the program in listing 5.3. We will show that, depending on whether the `boolean` variable `c` is initialized to `false` or `true`, the set of executions will change. Consequently, this will result in different happens-before order sets.

```
// Initial values
boolean c=false; // version 1
//boolean c=true; // version 2
int x=0;y=0;

// Threads definition
Thread one = new Thread(() -> {
  if(c) { // (1)
    x=1;  // (2)
  }
}).start();
Thread other = new Thread(() -> {
  y=x;    // (3)
}).start();
```

Listing 5.3: Program with a conditional statement.

Consider version 1 of the program in listing 5.3. That is, the case where variable `c` is initialized to `false`. We have the following happens-before order sets from program order and thread start rules:

$$HB^{m}_{po} = m(init(c)) \rightarrow m(init(x)) \rightarrow m(init(y)) \rightarrow m(start(t1)) \rightarrow m(start(t2))$$
$$HB_{init} = \{m(start(t1)) \rightarrow t1(1), m(start(t2)) \rightarrow t2(3)\}$$

Note that there are no sets for HB^{t1}_{po} and HB^{t2}_{po}. This is not an error.

The set HB^{t2}_{po} is empty simply because the $t2$ only contains one operation.

The explanation why HB^{t1}_{po} is empty is more subtle; as it is a consequence of the conditional statement. First, note that HB^{m}_{po} and HB_{init} hold for all executions. This is because all synchronization orders must consistent with the pairs of actions in those sets. Due to this, we can show that $m(init(c)) \rightarrow t1(1)$:

$$\begin{aligned} &m(init(c)) \rightarrow m(start(t1)) && \text{in } HB_{po} \\ &m(start(t1)) \rightarrow t1(1) && \text{in } HB_{init}. \end{aligned}$$

Since there are no other write access on variable `c`. This implies that, for all executions, $t1(1)$ will read the value `false`. As a consequence, action $t1(2)$ is not executed in any valid execution of the program. Since $t1(2)$ is not present in any execution of the program, then there cannot be happens-before pairs including it. This results in $t1$ effectively only executing one operation in all executions of the program.

In summary, both, HB^{t1}_{po} and HB^{t2}_{po}, are empty, as any execution in version 1 only includes one operation for these threads.

Consider now version 2 of the program in listing 5.3. That is, the case where variable `c` is initialized to `true`. We can use the same reasoning as above to show that $t1(1)$ reads the value `true` for variable `c` in all executions. Therefore, action $t1(2)$ is executed in all executions, which implies that $HB^{t1}_{po} = t1(1) \rightarrow t1(2)$ holds for all executions. □